Structure in C

A **structure** in C is a user-defined data type that allows us to group **different types of data under a single name**.

For example, information about a student may contain:

- Roll number → `int`
- Name → `char`
- Marks → `float`

These different types of data can be combined using a structure.

A structure is a user-defined data type in C that groups related variables of different data types under a single name.

Example

```
struct Student
{
    int roll_no;
    char name[30];
    float marks;
};
```

Here, `Student` is a structure containing three members: `roll_no`, `name`, and `marks`.

Defining a Structure

A structure is defined using the **`struct` keyword**.

Syntax

```
struct structure_name
{
    data_type member1;
    data_type member2;
    data_type member3;
};
```

Example

```
struct Student
{
    int roll_no;
    char name[30];
    float marks;
};
```

Explanation

- `struct` → keyword used to define a structure.
- `Student` → name of the structure.

- `roll_no`, `name`, and `marks` → structure members.
- The semicolon `;` after `}` is mandatory.

Important: Defining a structure does not allocate memory for its members. Memory is allocated when structure variables are declared.

Declaring Structure Variables

After defining a structure, we can declare variables of that structure type.

Syntax

```
struct structure_name variable_name;
```

Example

```
struct Student s1;
```

Here, `s1` is a variable of type `struct Student`.

We can also declare multiple variables at once:

```
struct Student s1, s2, s3;
```

Structure Definition and Declaration Together

A structure variable can also be declared while defining the structure:

```
struct Student
{
    int roll_no;
    char name[30];
    float marks;
} s1, s2;
```

Here, `s1` and `s2` are structure variables.

Initialization of a Structure

Structure initialization means assigning initial values to the members of a structure at the time of declaration.

Syntax

```
struct structure_name variable = {value1, value2, value3};
```

Example

```
struct Student
{
    int roll_no;
    char name[30];
    float marks;
```

```
};

struct Student s1 = {101, "Rahul", 85.5};
```

Here:

Member	Value
roll_no	101
name	"Rahul"
marks	85.5

The values are assigned according to the **order of members** in the structure.

Designated Initialization

C also allows members to be initialized by specifying their names:

```
struct Student s1 = {
    .roll_no = 101,
    .marks = 85.5,
    .name = "Rahul"
};
```

This method makes the initialization clearer and does not require following the member order.

Accessing Structure Members

The members of a structure are accessed using the **dot (.) operator**.

Syntax

```
structure_variable.member_name;
```

Example

```
printf("%d", s1.roll_no);
printf("%s", s1.name);
printf("%.2f", s1.marks);
```

To assign values:

```
s1.roll_no = 101;
s1.marks = 90.5;
```

For a character array:

```
strcpy(s1.name, "Rahul");
```

Complete Example

```
#include <stdio.h>
#include <string.h>

struct Student
{
```

```
    int roll_no;
    char name[30];
    float marks;
};

int main()
{
    struct Student s1;

    s1.roll_no = 101;
    strcpy(s1.name, "Rahul");
    s1.marks = 85.5;

    printf("Roll No: %d\n", s1.roll_no);
    printf("Name: %s\n", s1.name);
    printf("Marks: %.2f\n", s1.marks);

    return 0;
}
```

Output

```
Roll No: 101
Name: Rahul
Marks: 85.50
```

Operations on Individual Structure Members

Individual members of a structure can be used in the same way as ordinary variables of their respective data types.

The following operations can be performed on structure members:

Assignment

Values can be assigned to individual members.

```
s1.roll_no = 101;
s1.marks = 90.5;
```

Reading Input

Individual members can be used with input functions.

```
scanf("%d", &s1.roll_no);
scanf("%f", &s1.marks);
```

For a string:

```
scanf("%s", s1.name);
```

Arithmetic Operations

Arithmetic operations can be performed on numeric members.

```
s1.marks = s1.marks + 5;
```

Other operations such as:

```
s1.marks * 2
s1.marks / 2
s1.marks - 5
```

are also possible.

Comparison

Individual numeric members can be compared using relational operators.

```
if (s1.marks > s2.marks)
{
    printf("Student 1 has higher marks");
}
```

We can use operators such as:

```
>    <    >=    <=    ==    !=
```

Note: Two complete structures cannot normally be compared directly using `==` or `!=` in C. Their individual members must be compared.

Increment and Decrement

Numeric structure members can be incremented or decremented.

```
s1.roll_no++;
s1.marks--;
```

Using Structure Members in Expressions

Structure members can be used in calculations and expressions.

```
float total;

total = s1.marks + s2.marks;
```

Here, `marks` is accessed individually from both structure variables.

Important Points for Exams

1. A structure is a **user-defined data type**.
2. It can store **different types of data** under one name.
3. The keyword **`struct`** is used to define a structure.
4. Structure members are accessed using the **dot (`.`) operator**.
5. Defining a structure does not itself create a structure variable.
6. Memory is allocated when a structure variable is declared.
7. Individual structure members can be assigned, read, calculated, and compared according to their data types.

8. A structure can contain members of different data types such as `int`, `float`, `char`, arrays, pointers, and even other structures.

Operations on Structures

A structure variable can be used in several operations. However, C does **not** allow all operations directly on complete structures.

Assignment of Structures

One structure variable can be assigned to another structure variable of the **same structure type**.

Example

```
#include <stdio.h>

struct Student
{
    int roll_no;
    float marks;
};

int main()
{
    struct Student s1 = {101, 85.5};
    struct Student s2;

    s2 = s1;

    printf("Roll No: %d\n", s2.roll_no);
    printf("Marks: %.2f\n", s2.marks);

    return 0;
}
```

Here:

```
s2 = s1;
```

copies the values of all members of `s1` into `s2`.

Important Point

Structure assignment is allowed when both variables belong to the **same structure type**.

Passing a Structure to a Function

A structure can be passed to a function as an argument.

```
#include <stdio.h>

struct Student
{
    int roll_no;
    float marks;
```

```
};

void display(struct Student s)
{
    printf("Roll No: %d\n", s.roll_no);
    printf("Marks: %.2f\n", s.marks);
}

int main()
{
    struct Student s1 = {101, 85.5};

    display(s1);

    return 0;
}
```

Here, the complete structure `s1` is passed to the function `display()`.

Returning a Structure from a Function

A function can also return a structure.

```
struct Student getStudent()
{
    struct Student s = {101, 85.5};
    return s;
}
```

The returned structure can be stored in another structure variable.

```
struct Student s1;

s1 = getStudent();
```

Comparison of Structures

C does **not** allow direct comparison of two complete structures using operators such as:

```
s1 == s2;   // Not allowed
s1 != s2;   // Not allowed
```

Instead, individual members must be compared.

```
if (s1.roll_no == s2.roll_no &&
    s1.marks == s2.marks)
{
    printf("Both structures contain equal values");
}
```

Important Point

Structures can be assigned, passed to functions, and returned from functions, but complete structures cannot be directly compared using relational or equality operators.

Structure Within Structure

A structure can contain another structure as one of its members. This is called a **nested structure** or **structure within structure**.

It is useful when one object contains another logically related object.

Example

Suppose a student has a date of birth. Instead of keeping day, month, and year directly inside `Student`, we can create a separate `Date` structure.

```
struct Date
{
    int day;
    int month;
    int year;
};

struct Student
{
    int roll_no;
    char name[30];
    struct Date dob;
};
```

Here, `dob` is a structure member of `Student`.

Accessing Members of Nested Structure

The dot operator is used twice.

Syntax

```
outer_structure.inner_structure.member;
```

Example

```
struct Student s1;

s1.dob.day = 15;
s1.dob.month = 8;
s1.dob.year = 2002;
```

Here:

- `s1` → Student structure variable
- `dob` → Date structure member
- `day`, `month`, `year` → members of Date

Complete Example

```
#include <stdio.h>

struct Date
{
    int day;
```

```
    int month;
    int year;
};

struct Student
{
    int roll_no;
    char name[30];
    struct Date dob;
};

int main()
{
    struct Student s1 = {101, "Rahul", {15, 8, 2002}};

    printf("Roll No: %d\n", s1.roll_no);
    printf("Name: %s\n", s1.name);
    printf("Date of Birth: %d-%d-%d\n",
           s1.dob.day,
           s1.dob.month,
           s1.dob.year);

    return 0;
}
```

Output

```
Roll No: 101
Name: Rahul
Date of Birth: 15-8-2002
```

Advantages

- Organizes complex data systematically.
- Makes programs easier to understand.
- Allows related information to be grouped logically.
- Useful for representing real-world objects.

Array of Structures

An **array of structures** is an array in which every element is a structure variable of the same type.

It is useful for storing information about **multiple objects**.

For example, if we want to store information about 100 students, creating 100 separate variables would be inconvenient. Instead, we can create an array of structures.

Syntax

```
struct structure_name array_name[size];
```

Example

```
struct Student
{
```

```
    int roll_no;
    char name[30];
    float marks;
};

struct Student s[5];
```

Here, `s` is an array that can store information about **5 students**.

Accessing Array of Structure Members

An individual structure is accessed using its array index, followed by the dot operator.

Syntax

```
array_name[index].member_name;
```

Example

```
s[0].roll_no = 101;
s[0].marks = 85.5;

s[1].roll_no = 102;
s[1].marks = 90.0;
```

Here:

- `s[0]` → first student
- `s[1]` → second student
- `.roll_no` → accesses roll number
- `.marks` → accesses marks

Complete Example

```
#include <stdio.h>

struct Student
{
    int roll_no;
    char name[30];
    float marks;
};

int main()
{
    struct Student s[3] = {
        {101, "Rahul", 85.5},
        {102, "Amit", 90.0},
        {103, "Priya", 88.5}
    };

    for (int i = 0; i < 3; i++)
    {
        printf("Roll No: %d\n", s[i].roll_no);
        printf("Name: %s\n", s[i].name);
        printf("Marks: %.2f\n\n", s[i].marks);
    }
```

```
    return 0;
}
```

Output

```
Roll No: 101
Name: Rahul
Marks: 85.50

Roll No: 102
Name: Amit
Marks: 90.00

Roll No: 103
Name: Priya
Marks: 88.50
```

Important Point

An array of structures is used when information about multiple objects of the same type needs to be stored.

Pointers to Structures

A **pointer to a structure** is a pointer variable that stores the address of a structure variable.

Example

```
struct Student
{
    int roll_no;
    float marks;
};

struct Student s1;
struct Student *ptr;

ptr = &s1;
```

Here:

- `s1` → structure variable
- `&s1` → address of `s1`
- `ptr` → pointer to structure
- `ptr` stores the address of `s1`

Accessing Structure Members Using a Pointer

There are two ways to access members through a structure pointer.

Method 1: Using `(*pointer).member`

```
(*ptr).roll_no = 101;
(*ptr).marks = 85.5;
```

The parentheses are important because `.` has higher precedence than `*`.

Method 2: Using Arrow (->) Operator

C provides a simpler way to access structure members through a pointer.

```
ptr->roll_no = 101;
ptr->marks = 85.5;
```

The arrow operator `->` is specifically used with **pointers to structures**.

Relationship

```
ptr->roll_no
```

is equivalent to:

```
(*ptr).roll_no
```

Complete Example

```
#include <stdio.h>

struct Student
{
    int roll_no;
    float marks;
};

int main()
{
    struct Student s1;
    struct Student *ptr;

    ptr = &s1;

    ptr->roll_no = 101;
    ptr->marks = 85.5;

    printf("Roll No: %d\n", ptr->roll_no);
    printf("Marks: %.2f\n", ptr->marks);

    return 0;
}
```

Output

```
Roll No: 101
Marks: 85.50
```

Why Are Structure Pointers Used?

Structure pointers are especially useful when:

1. A large structure needs to be passed to a function efficiently.
2. Dynamic memory allocation is required.

3. We need to modify the original structure inside a function.
4. Data structures such as linked lists, trees, and graphs are implemented.

For example, linked lists heavily use pointers to structures.

Quick Revision Table

Topic	Main Concept	Example
Operations on Structures	Assignment, function passing, returning	`s2 = s1;`
Structure within Structure	One structure as a member of another	`s1.dob.day`
Array of Structures	Multiple structures stored in an array	`s[0].marks`
Pointer to Structure	Pointer stores address of structure	`ptr = &s1;`
Arrow Operator	Access member through structure pointer	`ptr->marks`

Most Important

Remember these four forms:

```
// Normal structure
s1.marks

// Nested structure
s1.dob.day

// Array of structures
s[0].marks

// Pointer to structure
ptr->marks
```

Union – Introduction

A **union** in C is a user-defined data type that allows different members of different data types to **share the same memory location**.

A union is similar to a structure, but there is one major difference:

- In a **structure**, each member has its own memory location.
- In a **union**, all members share the same memory location.

A union is a user-defined data type in C in which all members share the same memory location, and only one member can hold a meaningful value at a time.

Example

```
union Data
{
    int i;
    float f;
    char ch;
};
```

The union `Data` contains three members:

- `i` → integer
- `f` → floating-point number
- `ch` → character

All three members use the **same memory location**.

Difference Between Structure and Union

This is an important exam question.

Structure	Union
Each member has separate memory.	All members share the same memory.
All members can contain valid values simultaneously.	Normally, only one member's value is meaningful at a time.
Size is generally related to the combined storage required by its members, with possible padding.	Size is generally determined by its largest member, with possible alignment padding.
Changing one member does not normally affect other members.	Writing to one member can affect the value represented by another member.
Used when all data items are required together.	Used when only one of several possible data items is needed at a time.

Simple Example

Consider:

```
struct Example
{
    int i;
    float f;
};
```

Both `i` and `f` can store values simultaneously.

But:

```
union Example
{
    int i;
    float f;
};
```

If we store a value in `i` and then store a value in `f`, the previous value of `i` should no longer be considered the active value.

Declaring a Union

A union is declared using the **`union` keyword**.

Syntax

```
union union_name
{
    data_type member1;
```

```
    data_type member2;
    data_type member3;
};
```

Example

```
union Data
{
    int i;
    float f;
    char ch;
};
```

Here, `Data` is the name of the union and `i`, `f`, and `ch` are its members.

Declaring Union Variables

After defining the union, variables can be declared as:

```
union Data d1;
```

Multiple variables can also be declared:

```
union Data d1, d2, d3;
```

Definition and Declaration Together

A union variable can also be declared while defining the union:

```
union Data
{
    int i;
    float f;
    char ch;
} d1, d2;
```

Here, `d1` and `d2` are union variables.

Accessing Union Members

Union members are accessed using the **dot (`.`) operator**, just like structure members.

Example

```
union Data d1;

d1.i = 100;

printf("%d", d1.i);
```

Similarly:

```
d1.f = 25.5;
printf("%f", d1.f);
```

and:

```
d1.ch = 'A';
printf("%c", d1.ch);
```

Important Point

Although all members share the same memory, the program should normally treat **the most recently stored member as the active member**.

Initialization of a Union

A union can be initialized when it is declared.

```
union Data d1 = {100};
```

The first member, `i`, is initialized with `100`.

In modern C, a specific member can also be initialized using a designated initializer:

```
union Data d1 = {.f = 25.5};
```

Here, `f` is initialized.

Usage of Unions

Unions are useful when a program needs to store **one value from several possible data types**, but not all values at the same time.

Saving Memory

The main advantage of a union is **memory efficiency**.

For example:

```
union Data
{
    int i;
    float f;
    char ch;
};
```

All members share memory rather than having separate storage for each member.

Representing Different Types of Data

Suppose a record may contain either:

- an integer value,
- a decimal value, or
- a character.

A union can represent this situation efficiently.

```
union Data
{
    int number;
    float decimal;
    char character;
};
```

Only the required member needs to be used at a particular time.

Memory-Constrained Applications

Unions are useful in applications where memory usage is important, such as:

- Embedded systems
- Device programming
- Operating-system programming
- Hardware-related programming

Representing Variant Data

A union can be used when an object may have **one of several possible forms**.

For example:

```
union Value
{
    int integer_value;
    float float_value;
    char character_value;
};
```

The program can use whichever type is required.

Operations on Unions

Operations on unions are mainly performed on their **individual members**.

Assignment to a Member

A value can be assigned to a union member.

```
union Data d1;

d1.i = 50;
```

Reading a Member

A union member can be read using the dot operator.

```
printf("%d", d1.i);
```

Modifying a Member

A member can be modified like an ordinary variable.

```
d1.i = d1.i + 10;
```

Taking the Address of a Union

The address of a union variable can be obtained using the address-of operator `&`.

```
union Data d1;

printf("%p", (void *)&d1);
```

All union members begin at the same storage location, so their addresses are related to the same union storage.

Assignment Between Union Variables

Two union variables of the **same union type** can be assigned to each other.

```
union Data d1, d2;

d1.i = 100;
d2 = d1;
```

The contents of `d1` are copied to `d2`.

Important

Complete unions cannot be meaningfully compared using operators such as:

```
d1 == d2;    // Not allowed
```

If comparison is required, compare the appropriate individual members.

Complete Example of Union

```
#include <stdio.h>

union Data
{
    int i;
    float f;
    char ch;
};

int main()
{
    union Data d;

    d.i = 100;
    printf("Integer: %d\n", d.i);

    d.f = 25.5;
```

```
    printf("Float: %.2f\n", d.f);

    d.ch = 'A';
    printf("Character: %c\n", d.ch);

    return 0;
}
```

Explanation

When:

```
d.i = 100;
```

the integer member contains the active value.

When:

```
d.f = 25.5;
```

the same memory is used for `f`.

When:

```
d.ch = 'A';
```

the same memory is used for `ch`.

Therefore, the union is useful when **only one type of value needs to be stored at a time**.

Enumerated Data Types

An **enumerated data type**, or **enum**, is a user-defined data type in C that consists of a set of **named integer constants**.

The keyword **`enum`** is used to define an enumeration.

An enumeration is a user-defined data type consisting of a set of named integer constants that represent a fixed set of possible values.

For example, the days of a week have a fixed set of possible values:

```
MONDAY
TUESDAY
WEDNESDAY
THURSDAY
FRIDAY
SATURDAY
SUNDAY
```

Instead of using numbers such as `0`, `1`, `2`, etc., we can use meaningful names.

Declaring an Enumeration

Syntax

```
enum enum_name
{
    constant1,
    constant2,
    constant3
};
```

Example

```
enum Week
{
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY
};
```

Here, `Week` is an enumeration type.

A variable can be declared as:

```
enum Week today;
```

Values of Enumeration Constants

By default, the first enumerated constant has the value **0**, and each subsequent constant increases by 1.

For example:

```
enum Week
{
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY
};
```

The values are conceptually:

Constant	Value
MONDAY	0
TUESDAY	1
WEDNESDAY	2
THURSDAY	3

You can also assign explicit values.

```
enum Status
{
    SUCCESS = 1,
```

```
    FAILURE = 0
};
```

Another example:

```
enum Level
{
    LOW = 1,
    MEDIUM = 5,
    HIGH = 10
};
```

After an explicitly assigned value, subsequent unassigned enumerators continue incrementing from that value.

Example of Enum

```
#include <stdio.h>

enum Week
{
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY
};

int main()
{
    enum Week today;

    today = WEDNESDAY;

    printf("Value of today = %d", today);

    return 0;
}
```

Output

```
Value of today = 2
```

Uses of Enumerated Data Types

Enumerations are useful when a variable can have a **limited and predefined set of values**.

Common examples

Days:

```
enum Day
{
    MON, TUE, WED, THU, FRI, SAT, SUN
};
```

Traffic signals:

```
enum Signal
{
    RED,
    YELLOW,
    GREEN
};
```

Status:

```
enum Status
{
    FAILED,
    PASSED
};
```

Advantages

1. **Improves readability** because meaningful names are used instead of numbers.
2. Makes programs easier to understand and maintain.
3. Useful for representing a fixed set of choices.
4. Reduces the use of unexplained numeric constants in programs.

Union vs Enumeration

These are completely different concepts:

Union	Enumeration
Defined using `union`.	Defined using `enum`.
Groups different data types sharing the same storage.	Defines named integer constants.
Mainly used for memory-efficient storage.	Mainly used for representing a fixed set of choices.
Example: `int`, `float`, `char` sharing storage.	Example: `RED`, `GREEN`, `BLUE`.

Storage Classes in C

A **storage class** in C defines the **scope, lifetime, visibility, and storage behavior** of a variable or function.

In simple words, a storage class tells us:

- **Where** a variable can be accessed.
- **How long** the variable remains in memory.
- **What its default initial value is.**
- Whether the variable has **local or external linkage**.

A storage class in C specifies the scope, lifetime, visibility, and storage characteristics of a variable or function.

C provides four commonly taught storage classes:

1. **Automatic (`auto`)**
2. **Register (`register`)**
3. **Static (`static`)**
4. **External (`extern`)**

Automatic Storage Class

The **automatic storage class** is the default storage class for variables declared inside a function or block.

It is represented by the keyword:

```
auto
```

However, the `auto` keyword is rarely written explicitly because local variables are automatic by default.

Syntax

```
auto data_type variable_name;
```

Example

```
void display()
{
    auto int x = 10;
    printf("%d", x);
}
```

This is equivalent to:

```
void display()
{
    int x = 10;
    printf("%d", x);
}
```

Characteristics of Automatic Variables

1. Scope

An automatic variable has **block scope**.

It can be accessed only within the block or function where it is declared.

```
void test()
{
    int x = 10;

    printf("%d", x);
}
```

`x` cannot be accessed outside `test()`.

2. Lifetime

An automatic variable exists only while the block or function is executing.

When the function ends, the variable's lifetime ends.

3. Default Initial Value

If an automatic variable is not initialized, its value is **indeterminate**.

```
void test()
{
    int x;
    printf("%d", x);   // indeterminate value
}
```

Therefore, local automatic variables should be initialized before use.

Register Storage Class

The **register storage class** is used to request that a variable be stored in a **CPU register**, if possible, for faster access.

It uses the keyword:

```
register
```

Syntax

```
register data_type variable_name;
```

Example

```
register int count;
```

A common use is for frequently accessed variables such as loop counters.

```
for (register int i = 0; i < 1000; i++)
{
    printf("%d ", i);
}
```

Characteristics of Register Variables

1. Faster Access

A register variable may be stored in a CPU register, which can make access faster.

2. Scope

Like automatic variables, a register variable generally has **block scope** when declared inside a block.

3. Lifetime

Its lifetime is limited to the execution of the block in which it is declared.

4. Default Value

An uninitialized register variable has an **indeterminate value**.

5. Address Cannot Be Requested

The address-of operator `&` cannot be applied to a register variable.

```
register int x = 10;

printf("%p", (void *)&x);   // Not allowed
```

This is because the variable may be stored in a CPU register rather than a memory location with an ordinary address.

Important Point

The `register` keyword is a **request to the compiler**, not a command. The compiler may choose to ignore the request.

Static Storage Class

The **static storage class** is used when a variable needs to **retain its value throughout the execution of the program**.

It uses the keyword:

```
static
```

Static variables are especially important when a function needs to remember a value between calls.

Example

```
#include <stdio.h>

void counter()
{
    static int count = 0;

    count++;

    printf("%d\n", count);
}

int main()
```

```
{
    counter();
    counter();
    counter();

    return 0;
}
```

Output

```
1
2
3
```

Why?

Normally, a local variable is created when the function is called and its value does not persist after the function returns.

But:

```
static int count = 0;
```

retains its value between function calls.

Characteristics of Static Variables

1. Lifetime

A static variable exists for the **entire execution of the program**.

2. Default Initial Value

If a static variable is not explicitly initialized, it is automatically initialized to **0**.

```
static int x;
```

Here:

```
x = 0
```

3. Retains Value

Its value is preserved between function calls.

4. Local Static Variable

A static variable declared inside a function has **block scope**, but its lifetime is the entire program.

```
void test()
{
    static int x = 0;
    x++;
```

```
}
```

The variable `x` can be accessed only inside `test()`, but it continues to exist between calls.

Static Global Variable

A static variable can also be declared outside all functions.

```
static int count = 0;
```

Such a variable has **file scope** and internal linkage, meaning it is not directly accessible from other source files.

This is useful when a global variable should be restricted to the current source file.

External Storage Class

The **external storage class** is used to refer to a global variable that is defined elsewhere, often in another source file.

It uses the keyword:

```
extern
```

Syntax

```
extern data_type variable_name;
```

Example

Suppose we have two files.

File 1: `file1.c`

```
int marks = 90;
```

File 2: `file2.c`

```
#include <stdio.h>

extern int marks;

int main()
{
    printf("%d", marks);

    return 0;
}
```

Here:

```
extern int marks;
```

tells the compiler that `marks` is defined elsewhere.

Characteristics of External Variables

1. Scope

An external/global variable can have **file scope**, and an `extern` declaration can make an existing external object available in another source file.

2. Lifetime

An external variable has a lifetime that extends throughout the **execution of the program**.

3. Default Initial Value

A global variable with static storage duration that is not explicitly initialized is initialized to **0**.

```
int x;
```

Conceptually:

```
x = 0
```

4. Used for Multiple Files

`extern` is particularly useful in **multi-file programs**, where a global variable is defined in one source file and used in another.

Difference Between the Four Storage Classes

Feature	`auto`	`register`	`static`	`extern`
Main purpose	Normal local variable	Request fast access	Retain value	Refer to external variable
Scope	Block	Block	Block or file	Depends on the declared object
Lifetime	Block execution	Block execution	Entire program	Entire program for the referenced object
Default value	Indeterminate	Indeterminate	0	0 for a global object if not initialized
Keyword	`auto`	`register`	`static`	`extern`
Can retain value between function calls?	No	No	**Yes**	Yes, for the referenced global object
Typical use	Local variables	Frequently accessed variables	Counters/state	Sharing globals between files

Simple Example of All Four

```
#include <stdio.h>

int global = 100;

void test()
{
    auto int a = 10;
    register int b = 20;
    static int c = 30;

    c++;

    printf("Auto = %d\n", a);
    printf("Register = %d\n", b);
    printf("Static = %d\n", c);
}

int main()
{
    extern int global;

    test();
    test();

    printf("External = %d\n", global);

    return 0;
}
```

The important behavior is that `c` retains its value between calls:

```
First call:   c = 31
Second call:  c = 32
```

Whereas the automatic and register variables are recreated for each function invocation.